Week 5 · Lightning Presentation

Monday, 11 November 2026

Strict Source-Grounding & Global Sensemaking

The Google NotebookLM Audit & GraphRAG

Presented by: Mohammad Salman Sunakal
Matriculation Number: 5019034

Course: Conversational Artificial Intelligence

Code link: https://github.com/thesalmxn/week5-notebooklm-graphrag-audit

Strict Source-Grounding & Global Sensemaking

Why naive RAG fails — and how NotebookLM + GraphRAG enforce truth

Two coupled failure modes

- Leaky grounding — LLM blends retrieved spans with parametric memory;
- hallucination happens WITH citations attached
- Global sensemaking collapse — top-k chunks can't answer
- corpus-wide questions ('recurring themes across all docs?')

The fix we audit

- NotebookLM — closed-world refusal +
- token-level citations to source
- GraphRAG — Leiden community summaries
- for corpus-level synthesis

Thesis: grounding = retrieval + REFUSAL + citation lineage

03_NotebookLM_RAG_Audit.py — Evaluation Pipeline

- AUDIT HARNESS — not a product demo; works on any grounded RAG endpoint
- 1. Citation verification — do cited spans actually support the claim?
- 2. Negative-constraint probes — does it REFUSE out-of-corpus queries?
- 3. Audio Overview fidelity — does the 2-host podcast invent facts?

```
                                        # --- real signatures from 03_NotebookLM_RAG_Audit.py ---
def run_audit(endpoint, queries, probes, audio_eps, sources):
    log = []; cite_stats = {"checked": 0, "supported": 0}

    # A. citation verifier: claim-level support (Jaccard token overlap)
    for q in queries:
        resp = endpoint.ask(q.text)
        for cite, score in verify_citations(resp, sources, log):
            cite_stats["checked"]   += 1
            cite_stats["supported"] += int(score >= 0.5)

    # B. negative-constraint probes — CORRECT answer is refusal
    refusal_stats = run_refusal_probes(endpoint, probes, log)

    # C. Audio Overview audit — flag unsupported sentences
    audio_stats = [audit_audio(ep, sources, log) for ep in audio_eps]

    return {"citation": cite_stats, "refusal": refusal_stats,
            "audio": audio_stats,   "log": log}
```

Empirical Verification — Audit Log

Probe Type	N	Pass Rate	Failure Caught
Citation support	28	86%	Decorative citations
Negative-constraint	25	0% → 100%	Parametric leakage / fabrication
Audio fidelity	3	67%	Creative synthesis → false facts

GraphRAG probe: 'recurring themes across all docs?'

- vector RAG top-k → 3 disjoint chunks, no synthesis
- Leiden community summary → 1 coherent theme answer ✓

Sample audit log (real output from audit_report())

```
                                [PROBE-07] q='Q3 revenue breakdown?'  → ANSWERED ✗ PARAMETRIC LEAK
[PROBE-12] q='CEO maiden name?'       → ANSWERED ✗ PARAMETRIC LEAK
[CITE]     claim='48% growth'   span=doc1/chunk#0 support=1.00 ✓
[CITE]     claim='q4 layoffs'   span=doc1/chunk#1 support=0.00 ✗ DECORATIVE
[AUDIO-E2] sent='Founded in 1998...' → UNSUPPORTED ✗ (hallucinated year)
```

Key Engineering Takeaway

Grounded RAG is not retrieval + generation.

It is retrieval + generation constrained by REFUSAL

+ a verifiable citation lineage.

Without a refusal path, citations are decoration.